VERİ YAPILARI

İsmail KADAYIF

Veri Yapılarına Giriş

- Veri yapısı, verilerin etkin bir biçimde işlenebilmesi için belleğe nasıl kaydedileceğini belirler
- Veri yapısı, iki temel hususu açıklayabilen mantıksal bir fikir olarak düşünülebilir
  - ✓ Veri belleğe nasıl yerleştirilecek
  - ✓ Bu veri üzerinde ne tür işlemler uygulanabilecek

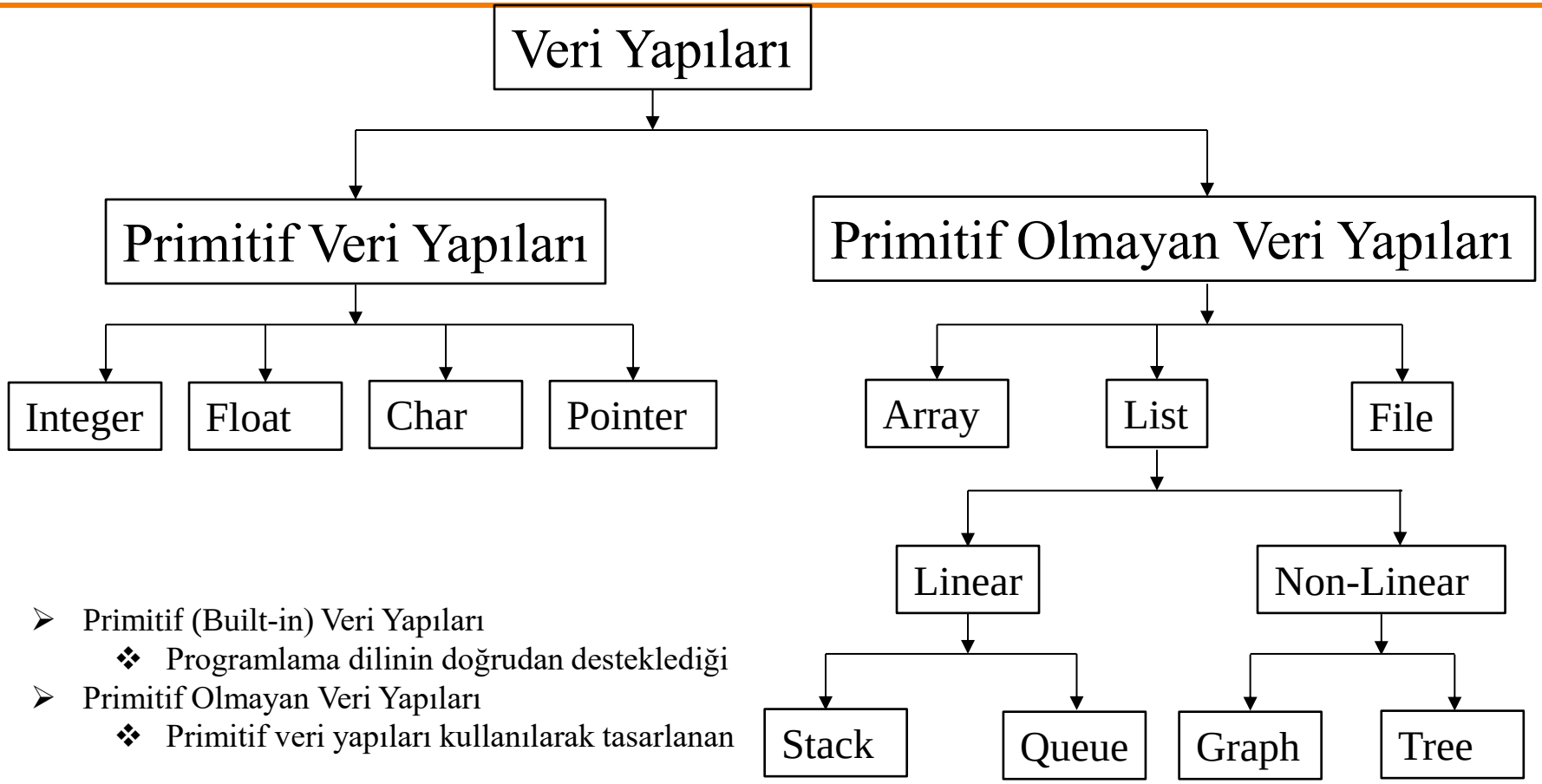
Veri Yapıları

- Bir algoritmaya ait programı gerçekleştirmek için o algoritmaya uygun veri yapısı (yapıları) seçmek gerekir
- Dolayısıyla program denince iki şey akla gelmeli
  - ✓ Algoritma
  - ✓ Veri Yapısı

Veri Yapılarının Sınıflandırılması

- Primitive (Built-in) Veri Yapıları
  - ✓ Üzerlerinde makine dili komutlarının direkt olarak çalışabildiği ve programlama dili tarafından direkt olarak desteklenen temel veri tipleri
  - ✓ Farklı bilgisayarlarda farklı temsilleri olabilir
  - ✓ int, float, double, char, string, pointer v.b.
- Non-Primitive Veri Yapıları
  - ✓ Primitive veri yapılarından türetilen daha karmaşık veri yapılarıdır
  - ✓ Aynı veya farklı veri birimlerinin ilişkilerle bir araya getirilmesiyle elde edilirler
  - ✓ Diziler, dosyalar, listeler, kuyruklar, v.b.

Veri Yapılarının Sınıflandırılması



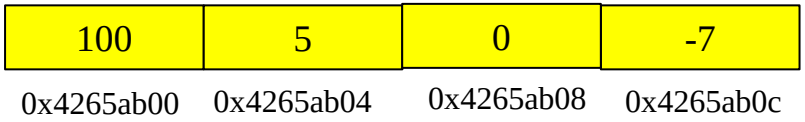
- Primitif (Built-in) Veri Yapıları
  - Programlama dilinin doğrudan desteklediği
- Primitif Olmayan Veri Yapıları
  - Primitif veri yapıları kullanılarak tasarlanan

Veri Organizasyonu

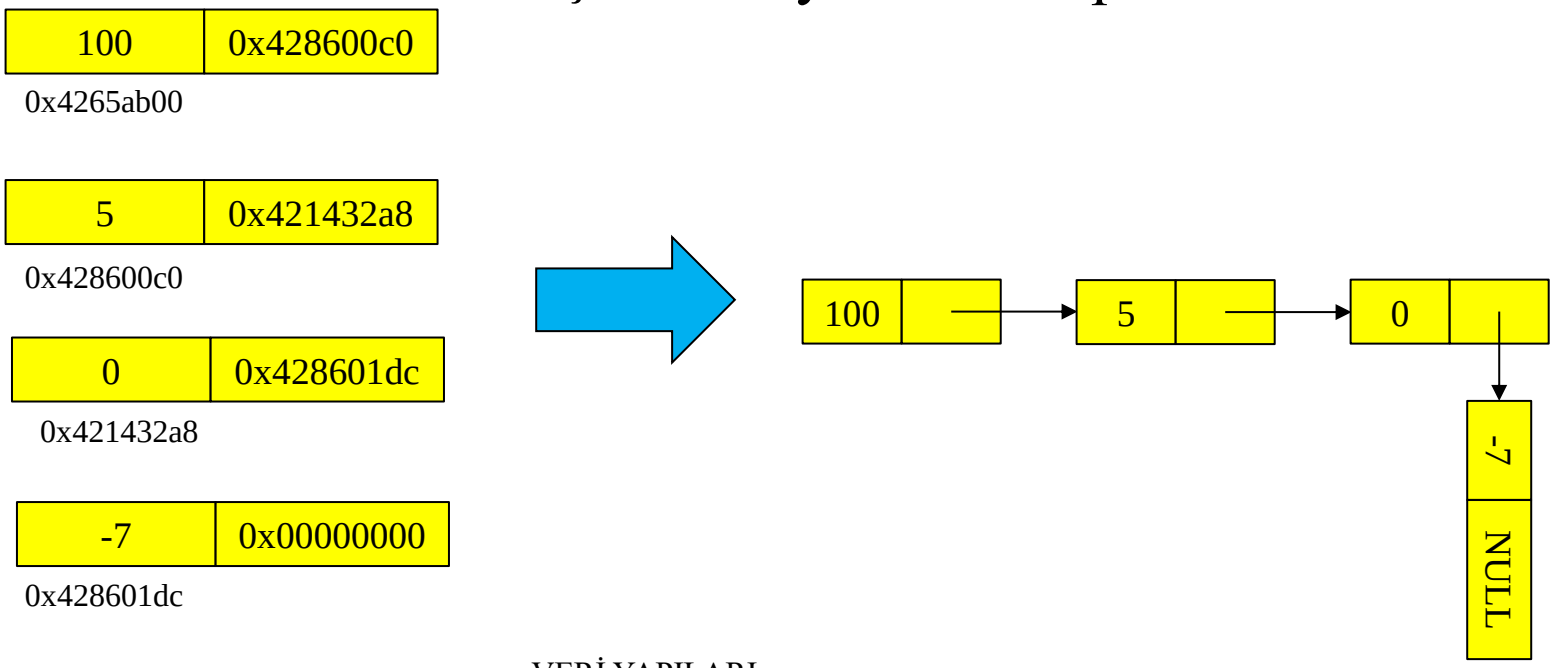
- Bir programda üzerinde çalışılan veri yapısının belleğe yerleşimi açısından belli bir yapısı veya organizasyonu vardır
- Kullanılan bir veri yapısı ne kadar karmaşık olursa olsun bellekteki organizasyonu açısından iki kategoriden birine girer
  - ✓ Contiguous (bitişik, sürekli)
    - ❖ Veri yapısını oluşturan veri parçaları bellekte bir arada tutulur
    - ❖ Diziler örnek olarak verilebilir ve dizilerin ardışık elemanları bellekte ardışık adreslerde tutulurlar
  - ✓ Non-contiguous (bitişik olmayan)
    - ❖ Veri yapısını oluşturan veri parçaları (bileşenler) belleğe dağıtılmış şekilde yerleştirilirler, bir parçadan diğerine linkler (pointerlar) vasıtasıyla geçiş yapılır. Bağlantılı listeler (link lists) bunlara örnek olarak verilebilir.

Veri Organizasyonu

Bitişik Veri Yapısı



Bitişik Olmayan Veri Yapısı

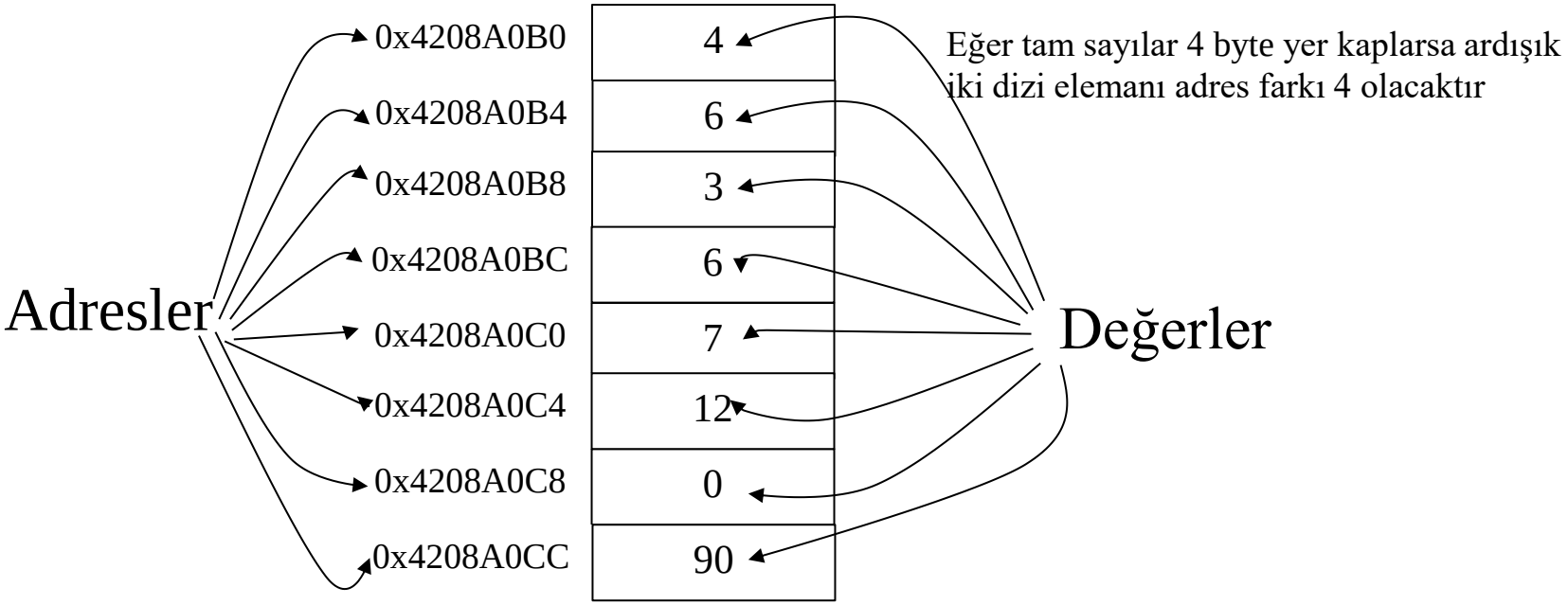


DİZİLER

- Bellekte statik olarak ayrılırlar
- C'deki dizilerin her bir elemanı aynı tipten olmak zorunda
- Dizi elemanları belleğe ardışık olarak yerleştirilir (bitişiklerdir)
- Dizi elemanlarına indeksleme yöntemiyle erişilir
- C'de dizinin ismi dizinin ilk elamanını gösteren sabit işaretçi (pointer) olarak düşünülür.
- Değişik boyutlarda olabilir
  - ✓ Bir boyutlu, iki boyutlu, üç boyutlu, dört boyutlu, …

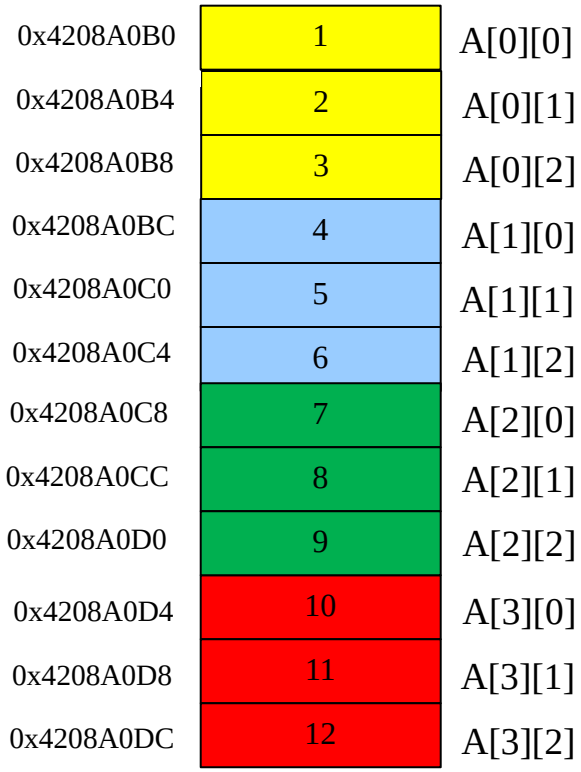
Bir boyutlu dizi

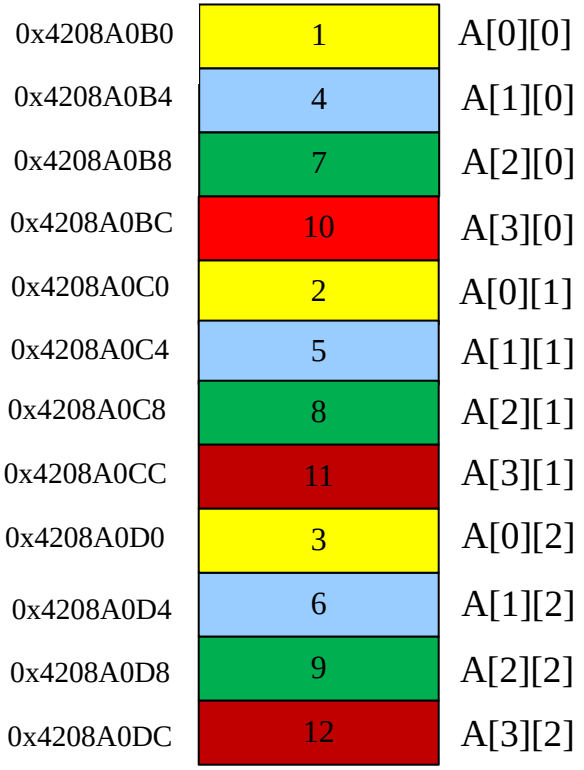
```
int A[8]={4,6,3,6,7,12,0,90}; // 8 elemanlı tamsayı dizisi
// int A[]={4,6,3,6,7,12,90,0}; Dizi elemanları, A[0],A[1], A[2],…., A[7]
// şeklinde de tanımlanabilir. Dizinin ilk elemanının indeksi 0'dır. (A[0])
```



İki boyutlu dizi

```
int A[4][3]={{1,2,3},{4,5,6}{7,8,9},{10,11,12}};
// int A[][3]={{1,2,3},{4,5,6}{7,8,9},{10,11,12}}; şeklinde de tanımlanabilir
// (ilk boyutun uzunluğunu belirtmemize gerek yok.
```

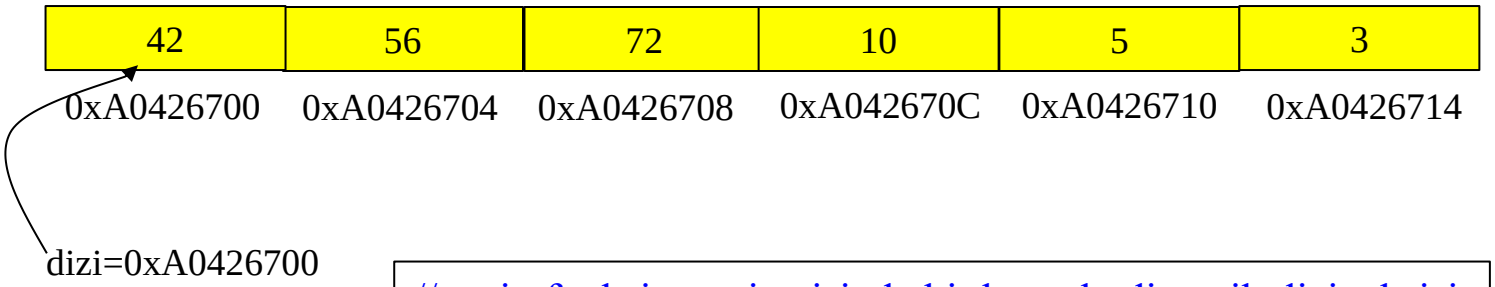




Row Major (C, Java, C++, …)

Column Major (Fortran)

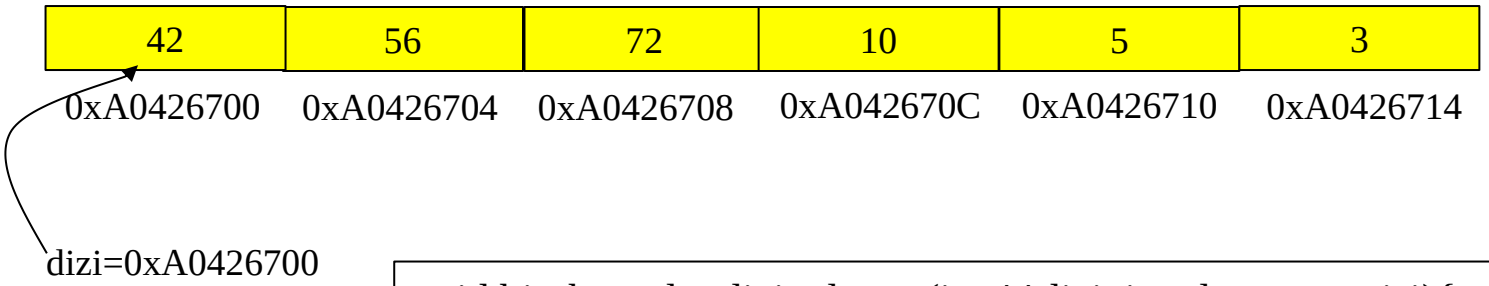
Bir boyutlu dinamik dizi tahsisi




```
// main fonksiyonu içerisinde bir boyutlu dinamik dizi tahsisi
int main(int argc, char** argv) {
   int *dizi;
   int eleman_sayisi = 6;
   dizi= (int*)malloc(sizeof(int)*eleman_sayisi);
   for(i=0; i<eleman_sayisi; i++){
      *(dizi+i)= rand()%100;
    }

   // 6 x 4 = 24 B lık bir bellek bloğu heapte tahsis edilir.
   // bu bellek bloğunun adresi dizi işaretçisine atanır.
   // Toplam alan gereksinimi: 4 + 6 x 4 = 28 Byte
```

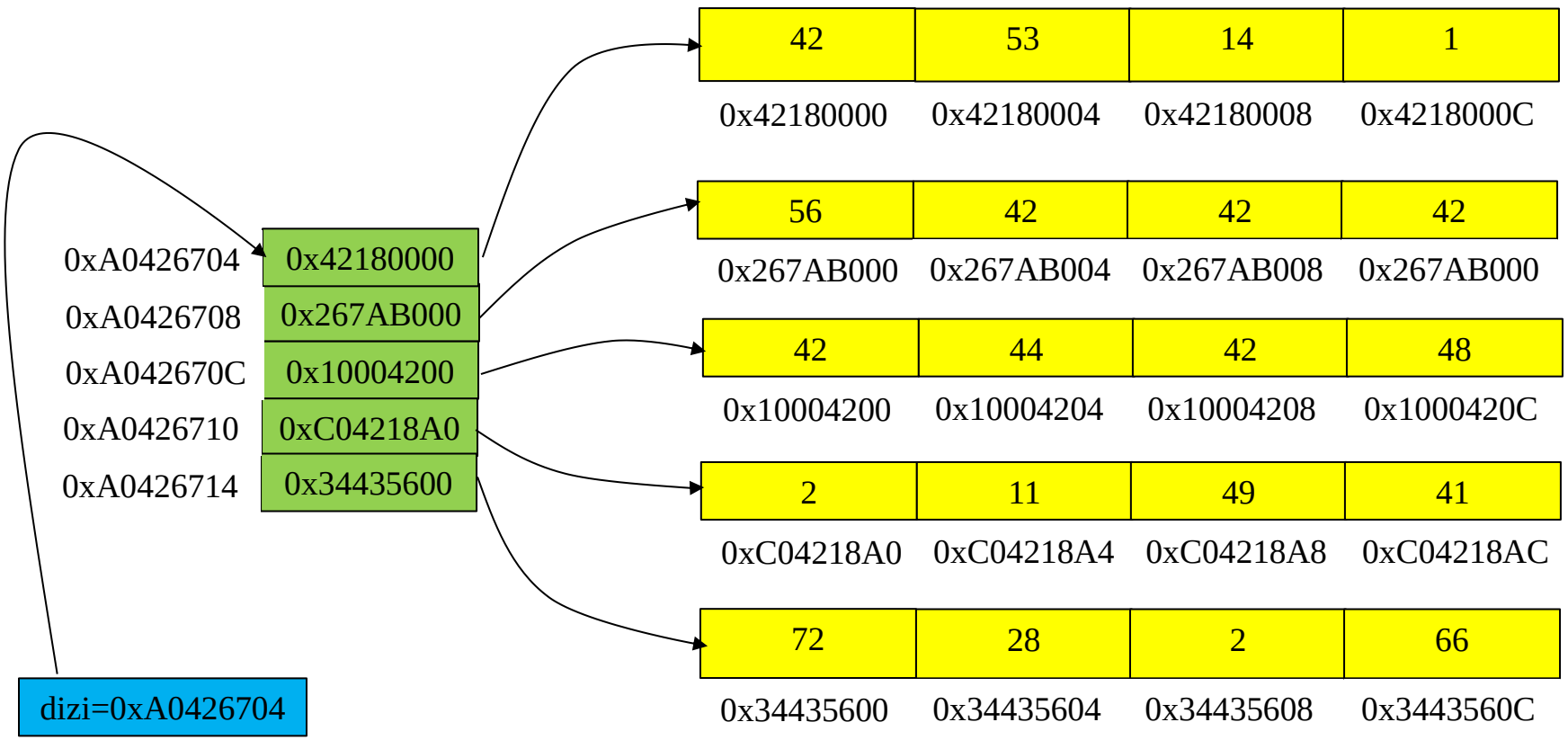
Bir boyutlu dinamik dizi tahsisi




```
void bir_boyutlu_dizi_olustur(int **dizi, int eleman_sayisi){
    int i;
    *dizi=(int*)malloc(sizeof(int)*eleman_sayisi);

    for(i=0; i<eleman_sayisi; i++){
        *(*dizi+i)= rand()%100;
    }
}
int main(int argc, char** argv) {
    int *dizi;
    int eleman_sayisi = 6;
    bir_boyutlu_dizi_olustur(&dizi, eleman_sayisi);
    // dizi değişkeninin değeri değiştirilmesi için adresi fonksiyona yollanmalı
}
```

İki boyutlu dinamik dizi tahsisi



İki boyutlu bir dizi aslında tek boyutlu bir dizi, öyle ki bu tek boyutlu dizinin her bir elemanı tek boyutlu bir dizi

İki boyutlu dinamik dizi tahsisi

```
// main fonksiyonu içerisinde iki boyutlu dinamik dizi tahsisi
int main(int argc, char** argv) {
int **dizi;
int satir_sayisi=5;
int sutun_sayisi =4;
dizi=(int**)malloc(satir_sayisi*sizeof(int*));
    for(i=0; i<satir_sayisi; i++){
        *(dizi+i)= (int*)malloc(sutun*sizeof(int));
        for(j=0; j<sutun_sayisi; j++)
            *(*(dizi+i)+j)= rand()%100;
    }
}
// Toplam alan gereksinimi: (4 + 5x4 + 5x4x4) Byte
```

İki boyutlu dinamik dizi tahsisi

```
void dinamik_matris_olustur(int ***dizi,int satir,int sutun){
    int i,j;
    *dizi=(int**)malloc(satir*sizeof(int*));
    for(i=0; i<satir; i++){
        *(*dizi+i)= (int*)malloc(sutun*sizeof(int));
        for(j=0; j<sutun; j++)
            *(*(*dizi+i)+j)= rand()%100;
    }
}

int main(int argc, char** argv) {
    int **dizi;
    int satir_sayisi=5;
    int sutun_sayisi=4;
    dinamik_matris_olustur(&dizi, satir_sayisi, sutun_sayisi);

   // dizi değişkeninin değeri değiştirilmesi için adresi fonksiyona yollanmalı
}
```